Count of Smaller Numbers After Self

Given an integer array `nums`, return an integer array `counts` where `counts`$[i]$ is the number of smaller elements to the right of `nums`$[i]$.

Example Test Cases:

TEST CASE 1
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.

TEST CASE 2
Input: nums = [-1]
Output: [0]
Explanation: No element exists to the right of -1.

TEST CASE 3
Input: nums = [-1,-1]
Output: [0,0]
Explanation: No element to the right of any position is strictly smaller.

Expected Time Complexity: $O(n \log n)$

Input Format:
An integer array `nums` of length n.

Output Format:
Return an integer array `counts` of length n, where `counts`$[i]$ represents the number of elements strictly smaller than `nums`$[i]$ appearing to its right.

Assumptions:

1. $1 \leq t \leq 1000$
2. $1 \leq n \leq 10^5$
3. Each element `nums`$[i]$ lies in the range $[-10^4, 10^4]$.
4. It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

(After reading this question, follow the steps described in the `Instructions.pdf` file to see the exact input and output format for the provided test cases.)

Instructions:

You have been provided with two template files, containing the `main` and `solve` functions respectively. The `main` function parses the input for each testcase. For each sub-test-case, you have to call the `solve` function, which computes and returns the required array `counts`.

The `solve` function must compute this in $O(n \log n)$ time.

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)

Library Usage:

For this lab, you are permitted to use any modules which are part of the standard library of either `C++` or `python`.